ETH Zürich

Informatik I
Übungsstunde 03

Johannes Odenthal

ETZ J 91
cs.jodenthal.com

Hausaufgaben

Fragen zu letzter Woche?

Exercise 2: Expressions, Fundamental Types

Task 1: Expressions
Task 2: Expression Evaluation
Task 3: Equivalent Resistance

Scopes

> Scopes: siehe C++-Guide auf der Kurswebseite.

For Loops

Die for-Schleife

```
for (init; condition; update) {
  // Schleifenrumpf
}
```

In welcher Reihenfolge läuft das ab?

Die for-Schleife

```
for (init; condition; update) {
  // Schleifenrumpf
}
```

> Initialisierung: einmal zu Beginn.

Die for-Schleife

```
for (init; condition; update) {
  // Schleifenrumpf
}
```

> Bedingung: vor jeder Iteration prüfen.

Die for-Schleife

```
for (init; condition; update) {
  // Schleifenrumpf
}
```

> Rumpf: nur bei wahrer Bedingung ausführen.

Die for-Schleife

```
for (init; condition; update) {
  // Schleifenrumpf
}
```

> Update: nach dem Rumpf. Dann die Bedingung erneut prüfen.

Eine leere Bedingung gilt als `true`.

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i

Was wird ausgegeben?

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	**0**	—

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	—
Init	`i = 1`	0	**1**

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	—
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	–
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1
Rumpf	`0 + 1 = 1`	**1**	1

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	–
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1
Rumpf	`0 + 1 = 1`	1	1
Update	`++i`	1	**2**

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	–
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1
Rumpf	`0 + 1 = 1`	1	1
Update	`++i`	1	2
Bedingung	`2 <= 3` : **true**	1	2

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	–
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1
Rumpf	`0 + 1 = 1`	1	1
Update	`++i`	1	2
Bedingung	`2 <= 3` : **true**	1	2
Rumpf	`1 + 2 = 3`	**3**	2

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	–
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1
Rumpf	`0 + 1 = 1`	1	1
Update	`++i`	1	2
Bedingung	`2 <= 3` : **true**	1	2
Rumpf	`1 + 2 = 3`	3	2
Update	`++i`	3	**3**

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	–
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1
Rumpf	`0 + 1 = 1`	1	1
Update	`++i`	1	2
Bedingung	`2 <= 3` : **true**	1	2
Rumpf	`1 + 2 = 3`	3	2
Update	`++i`	3	3
Bedingung	`3 <= 3` : **true**	3	3

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	–
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1
Rumpf	`0 + 1 = 1`	1	1
Update	`++i`	1	2
Bedingung	`2 <= 3` : **true**	1	2
Rumpf	`1 + 2 = 3`	3	2
Update	`++i`	3	3
Bedingung	`3 <= 3` : **true**	3	3
Rumpf	`3 + 3 = 6`	**6**	3

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	–
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1
Rumpf	`0 + 1 = 1`	1	1
Update	`++i`	1	2
Bedingung	`2 <= 3` : **true**	1	2
Rumpf	`1 + 2 = 3`	3	2
Update	`++i`	3	3
Bedingung	`3 <= 3` : **true**	3	3
Rumpf	`3 + 3 = 6`	6	3
Update	`++i`	6	**4**

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

Die Bedingung ist falsch. Wir verlassen die Schleife.

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	—
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1
Rumpf	`0 + 1 = 1`	1	1
Update	`++i`	1	2
Bedingung	`2 <= 3` : **true**	1	2
Rumpf	`1 + 2 = 3`	3	2
Update	`++i`	3	3
Bedingung	`3 <= 3` : **true**	3	3
Rumpf	`3 + 3 = 6`	6	3
Update	`++i`	6	4
Bedingung	`4 <= 3` : **false**	6	—

Tracing: Was passiert als Nächstes?

```
int sum = 0;
for (int i = 1; i <= 3; ++i) {
  sum += i;
}
std::cout << sum << "\n";
```

`i` ist jetzt ausserhalb seines Scopes.

Schritt	Auswertung	sum	i
Init	`sum = 0`	0	—
Init	`i = 1`	0	1
Bedingung	`1 <= 3` : **true**	0	1
Rumpf	`0 + 1 = 1`	1	1
Update	`++i`	1	2
Bedingung	`2 <= 3` : **true**	1	2
Rumpf	`1 + 2 = 3`	3	2
Update	`++i`	3	3
Bedingung	`3 <= 3` : **true**	3	3
Rumpf	`3 + 3 = 6`	6	3
Update	`++i`	6	4
Bedingung	`4 <= 3` : **false**	6	—
Ausgabe	`6` + Newline	6	—

Strange Sum

Lies eine positive ganze Zahl `n` ein. Addiere alle ungeraden Zahlen von 1 bis einschliesslich `n`, die nicht durch 5 teilbar sind. Gib die Summe aus.

Eingabe

```
10
```

Ausgabe

```
20
```

> Erst selbst auf Papier überlegen.

Strange Sum

Lies eine positive ganze Zahl `n` ein. Addiere alle ungeraden Zahlen von 1 bis einschliesslich `n`, die nicht durch 5 teilbar sind. Gib die Summe aus.

Eingabe

```
10
```

Ausgabe

```
20
```

> Austauschen und zu zweit programmieren.

Largest Power

Lies eine positive ganze Zahl `n` ein. Gib die grösste Zweierpotenz aus, die nicht grösser als `n` ist. Zweierpotenzen sind 1, 2, 4, 8, 16, …

Eingabe	Ausgabe
13	8

> Erst selbst auf Papier überlegen.

Largest Power

Lies eine positive ganze Zahl `n` ein. Gib die grösste Zweierpotenz aus, die nicht grösser als `n` ist. Zweierpotenzen sind 1, 2, 4, 8, 16, …

Eingabe

```
13
```

Ausgabe

```
8
```

> Austauschen und zu zweit programmieren.

09:00–09:15

15 Minuten Pause

Weiter um 09:15

Largest Power

Largest Power: Weiterarbeiten

Lies eine positive ganze Zahl `n` ein. Gib die grösste Zweierpotenz aus, die nicht grösser als `n` ist. Zweierpotenzen sind 1, 2, 4, 8, 16, …

Eingabe	Ausgabe
13	8

> Partnerarbeit fortsetzen.

Fragen?

Debugging

Live Demo

`non_terminating.cpp`

Fragen?

Repetition

Repetition

`5 < 4 < 1`

`true > false`

Notiere die Zwischenschritte.

Repetition: Schritt für Schritt

`5 < 4 < 1`

Repetition: Schritt für Schritt

`5 < 4 < 1`

`(5 < 4) < 1`

Repetition: Schritt für Schritt

```
5 < 4 < 1
```

```
(5 < 4) < 1
```

```
false < 1
```

Repetition: Schritt für Schritt

```
5 < 4 < 1
```

```
(5 < 4) < 1
```

```
false < 1
```

```
0 < 1
```

Repetition: Schritt für Schritt

`5 < 4 < 1`

`(5 < 4) < 1`

`false < 1`

`0 < 1`

`true`

Repetition: Schritt für Schritt

`5 < 4 < 1`

`(5 < 4) < 1`

`false < 1`

`0 < 1`

`true`

> Bereichstests brauchen zwei Vergleiche mit `&&`.

Repetition: Schritt für Schritt

`5 < 4 < 1`

`(5 < 4) < 1`

`false < 1`

`0 < 1`

`true`

`true > false`

> Bereichstests brauchen zwei Vergleiche mit `&&`.

Repetition: Schritt für Schritt

`5 < 4 < 1`

`(5 < 4) < 1`

`false < 1`

`0 < 1`

`true`

`true > false`

`true > 0`

> Bereichstests brauchen zwei Vergleiche mit `&&`.

Repetition: Schritt für Schritt

```
5 < 4 < 1
```

```
(5 < 4) < 1
```

```
false < 1
```

```
0 < 1
```

```
true
```

```
true > false
```

```
true > 0
```

```
1 > 0
```

> Bereichstests brauchen zwei Vergleiche mit `&&`.

Repetition: Schritt für Schritt

`5 < 4 < 1`

`(5 < 4) < 1`

`false < 1`

`0 < 1`

`true`

`true > false`

`true > 0`

`1 > 0`

`true`

> Bereichstests brauchen zwei Vergleiche mit `&&`.

Fragen?

Floating Point Numbers

Floating Point: Gleichheit nach Rundung

```
float a = 2.0f;
if (std::sqrt(2.0f) * std::sqrt(2.0f) == a) {
  std::cout << "A";
}
```

Was ist die Ausgabe?

Floating Point: Gleichheit nach Rundung

```
float a = 2.0f;
if (std::sqrt(2.0f) * std::sqrt(2.0f) == a) {
  std::cout << "A";
}
```

Was ist die Ausgabe?

Bedingung `false`. Keine Ausgabe.

Floating Point: Gleichheit nach Rundung

```
float a = 2.0f;
if (std::sqrt(2.0f) * std::sqrt(2.0f) == a) {
  std::cout << "A";
}
```

Was ist die Ausgabe?

Bedingung `false`. Keine Ausgabe.

Mathematisch gilt $\left(\sqrt{2}\right)^2 = 2$. $\sqrt{2}$ ist nicht exakt darstellbar; auch das Produkt wird gerundet.

> `==` nach unterschiedlichen Berechnungen oder Rundung vermeiden.

Floating Point: Gross + klein

```
float a = 67108864.0f + 0.1f;
if (a > 67108864.0f) {
  std::cout << "A";
}
```

Was ist die Ausgabe?

Floating Point: Gross + klein

```
float a = 67108864.0f + 0.1f;
if (a > 67108864.0f) {
  std::cout << "A";
}
```

Was ist die Ausgabe?

`a` **bleibt** `67108864.0f`**. Keine Ausgabe.**

Floating Point: Gross + klein

```
float a = 67108864.0f + 0.1f;
if (a > 67108864.0f) {
  std::cout << "A";
}
```

Was ist die Ausgabe?

`a` bleibt `67108864.0f`. Keine Ausgabe.

> Addition von Zahlen sehr unterschiedlicher Grösse vermeiden.

Rundung: Der kleine Beitrag fehlt

```
67108864   = 6.7108864  * 10^7
       0.1 = 0.00000001 * 10^7
```

Rundung: Der kleine Beitrag fehlt

```
67108864   = 6.7108864  * 10^7
       0.1 = 0.00000001 * 10^7
```

Exakte Summe

```
67108864.1 = 6.71088641 * 10^7
```

Rundung: Der kleine Beitrag fehlt

```
67108864   = 6.7108864  * 10^7
       0.1 = 0.00000001 * 10^7
```

Exakte Summe

```
67108864.1 = 6.71088641 * 10^7
```

Nach Rundung

```
67108864   = 6.7108864  * 10^7
```

Begrenzte signifikante Stellen · Dezimalskizze zur Intuition

Fragen?

Comparing Floating Point Numbers

Vergleichen mit Toleranz

Für `float` und `double`: kleine Abweichungen zulassen.

$$c > 0 \quad |\mathrm{fp1} - \mathrm{fp2}| < c$$

Mit $c = 0.001$:

Vergleichen mit Toleranz

Für `float` und `double`: kleine Abweichungen zulassen.

$$c > 0 \quad |\mathrm{fp1} - \mathrm{fp2}| < c$$

Mit $c = 0.001$:

$|10.0 - 12.0| = 2.0$; **nicht nahe beieinander.**

Vergleichen mit Toleranz

Für `float` und `double`: kleine Abweichungen zulassen.

$$c > 0 \quad |\mathrm{fp1} - \mathrm{fp2}| < c$$

Mit $c = 0.001$:

$|10.0 - 12.0| = 2.0$; **nicht nahe beieinander.**

$|10.0 - 10.000013| \approx 0.000013$; **nahe beieinander.**

equals implementieren

```
// POST: true genau dann, wenn |x - y| < tol
bool equals(double x, double y, double tol) {
  // ...
}
```

Funktionsaufruf

```
equals(1.0, 1.0005, 0.001)
```

Rückgabewert

```
true
```

Wie berechnest du den Betrag der Differenz?

equals: Eine Lösung

```
bool equals(double x, double y, double tol) {
  double diff = x - y;
  if (diff < 0) {
    diff *= -1;
  }
  return diff < tol;
}
```

equals: Eine Lösung

```
bool equals(double x, double y, double tol) {
  double diff = x - y;
  if (diff < 0) {
    diff *= -1;
  }
  return diff < tol;
}
```

Eine feste absolute Toleranz kann für grosse Werte ungeeignet sein.

Fragen?

cs.jodenthal.com

Johannes Odenthal

Optional & Nachschlagen

- Debugging in CodeExpert
- Even Numbers
- Befehlsübersicht Woche 03

Optional: Debugging in CodeExpert

Finde die grösste positive und die kleinste negative Zweierpotenz, die in `int` passen. Gib beide Zahlen aus.

Eingabe

```
keine
```

Ausgabe

```
1073741824 -2147483648
```

Beispielausgabe für einen 32-Bit-`int`.

Optional: Einrückung reicht nicht

Absichtlich fehlerhaftes Beispiel aus einer Schleife.

```
if (condition)
  std::cout << "Stopping...\n";
  break;
```

Wann wird break ausgeführt?

Optional: Vorher prüfen

Absichtlich fehlerhafte Prüfungen vor dem Verdoppeln.

```
if (start * 2 > INT_MAX) {
  break;
}
if (start * 2 < INT_MIN) {
  break;
}
```

Was kann schon im Test schiefgehen?

Optional: Even Numbers

Lies eine ganze Zahl `n` ein. Gib die grösste gerade Zahl aus, die nicht grösser als `n` ist. Wenn `n` negativ ist, gib 0 aus.

Eingabe	Ausgabe
15	14

> Erst selbst auf Papier überlegen.

Optional: Even Numbers

Lies eine ganze Zahl `n` ein. Gib die grösste gerade Zahl aus, die nicht grösser als `n` ist. Wenn `n` negativ ist, gib 0 aus.

Eingabe	Ausgabe
15	14

> Austauschen und zu zweit programmieren.

Nachschlagen: for

```
int n;
std::cin >> n;
// Summe 1 + 2 + 3 + ... + n
int sum = 0;
for (int i = 1; i <= n; ++i) {
  sum += i;
}
std::cout << "1 + 2 + ... + n = " << sum << "\n";
```

Eine leere Bedingung gilt als `true`.

Nachschlagen: if–else

```
int a;
int b;
std::cin >> a >> b;
if (a < b) {
  std::cout << 3 << "\n";
} else {
  std::cout << 8 << "\n";
}
```

`else` ist optional. `{ ... }` gruppiert Anweisungen.

Nachschlagen: Scope

```
int main() {
  int a;
  std::cin >> a;
  if (a < 4) {
    std::cout << a << " "; // a im inneren Block gültig
    int b = 18;
    std::cout << b << " "; // b hier gültig
  } else {
    std::cout << b << " "; // Fehler: vor Deklaration
    int b = 11;
  }
  std::cout << a << " ";   // a weiterhin gültig
  std::cout << b << "\n"; // Fehler: ausserhalb des Scopes
  return 0;
}
```

Nachschlagen: Scope

```
int main() {
  int a;
  std::cin >> a;
  if (a < 4) {
    std::cout << a << " "; // a im inneren Block gültig
    int b = 18;
    std::cout << b << " "; // b hier gültig
  } else {
    std::cout << b << " "; // Fehler: vor Deklaration
    int b = 11;
  }
  std::cout << a << " ";   // a weiterhin gültig
  std::cout << b << "\n"; // Fehler: ausserhalb des Scopes
  return 0;
}
```

Scope: ab Deklaration bis zum Ende des Blocks.